

```

"""
Simple User Login System
-----
NOTE: This code contains INTENTIONAL
security flaws for educational
purposes (CodSoft Cyber Security Internship
- Task 3: Secure Code
Assessment). Do NOT use this code in a real
application.
"""

import sqlite3

# --- Flaw 1: Hardcoded credentials /
secret key directly in source code ---
ADMIN_PASSWORD = "admin123"
SECRET_API_KEY =
"sk_live_51Hy8h2KZ9d8fA0bC1x2Y3z4"

DB_FILE = "users.db"

def create_database():
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS users (
            username TEXT,
            password TEXT
        )
    """)

```

```

    """)
    conn.commit()
    conn.close()

def register_user(username, password):
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    # --- Flaw 2: Storing passwords in
    plain text (no hashing) ---
    cursor.execute(
        f"INSERT INTO users (username,
password) VALUES ('{username}',
'{password}')"
    )
    conn.commit()
    conn.close()
    print(f"User '{username}' registered
successfully.")

def login_user(username, password):
    conn = sqlite3.connect(DB_FILE)
    cursor = conn.cursor()

    # --- Flaw 3: SQL Injection
    vulnerability ---
    # User input is directly inserted into
    the SQL query string instead
    # of using parameterized queries. An
    attacker could type something
    # like: ' OR '1'='1
    # as the username or password and log

```

```

in without knowing any
    # real credentials.
    query = f"SELECT * FROM users WHERE
username = '{username}' AND password =
'{password}'"
    cursor.execute(query)
    result = cursor.fetchone()
    conn.close()

    if result:
        print("Login successful!")
        return True
    else:
        print("Login failed.")
        return False

def admin_login(password):
    # --- Flaw 4: Insecure comparison +
hardcoded password check ---
    if password == ADMIN_PASSWORD:
        print("Welcome, admin!")
        return True
    print("Access denied.")
    return False

def get_user_input_and_login():
    # --- Flaw 5: No input validation ---
    # There's no check on length, allowed
characters, or empty input
    # before this data is used in a
database query.
    username = input("Enter username: ")

```

```
password = input("Enter password: ")  
login_user(username, password)
```

```
if __name__ == "__main__":  
    create_database()  
    register_user("testuser",  
"password123")  
    get_user_input_and_login()
```